

Nexus Inventory

Definición de arquitectura frontend backend

1. Introducción

Este documento define la **arquitectura general frontend-backend** del sistema **Nexus Inventory**.

El objetivo es establecer cómo se organiza la aplicación, cómo se comunican sus partes y qué responsabilidades tiene cada capa, permitiendo un desarrollo ordenado y escalable.

2. Visión general de la arquitectura

El sistema se basa en una **arquitectura cliente-servidor**, separando claramente:

- **Frontend**: interfaz que usa el usuario desde el navegador
- **Backend**: lógica del sistema y gestión de datos
- **Base de datos**: almacenamiento persistente

La comunicación entre frontend y backend se realiza mediante una **API REST** usando HTTP/HTTPS.

3. Arquitectura Frontend

Función

El frontend es la parte visible del sistema y permite al usuario interactuar con la aplicación.

Características

- Aplicación web accesible desde un navegador
- Desplegada en la nube (AWS Amplify en el futuro)
- Consume datos del backend mediante peticiones HTTP

Responsabilidades

- Mostrar pantallas (login, productos, ventas, usuarios)
- Enviar peticiones al backend
- Mostrar respuestas y errores
- No contiene lógica de negocio crítica

4. Arquitectura Backend

Función

El backend gestiona la **lógica de negocio**, la **seguridad** y el **acceso a la base de datos**.

Características

- Servidor de aplicación desplegado en AWS
- Expone una API REST
- Procesa peticiones del frontend

Responsabilidades

- Autenticación y autorización
- Gestión de usuarios, productos y ventas
- Validación de datos
- Comunicación con la base de datos

5. Comunicación Frontend – Backend (API REST)

La comunicación se realiza mediante una **API REST**, usando:

- **HTTP methods:**
 - GET (obtener datos)
 - POST (crear datos)

- PUT (actualizar datos)
- DELETE (eliminar datos)
- **Formato de datos: JSON**

Ejemplo de flujo:

1. El usuario accede al frontend
2. El frontend envía una petición al backend
3. El backend procesa la petición
4. El backend responde con datos o errores
5. El frontend muestra la respuesta

6. Estructura general de servicios

- **Cliente (navegador)**
- **Frontend Web**
- **Backend (API REST)**
- **Base de datos en la nube (AWS)**

Cada componente está desacoplado, lo que facilita mantenimiento y escalabilidad.

7. Diagrama de arquitectura (descripción)

El diagrama de alto nivel incluye:

- Usuario accediendo al frontend
- Frontend comunicándose con el backend mediante API REST
- Backend conectado a la base de datos
- Todo desplegado en infraestructura cloud (AWS)

8. Decisiones arquitectónicas

- Separación clara entre frontend y backend
- Uso de API REST para la comunicación
- Arquitectura preparada para crecer
- Compatible con servicios cloud y contenedores

9. Conclusión

Esta arquitectura permite un desarrollo estructurado, escalable y fácil de mantener.
El documento sirve como guía para futuras implementaciones del sistema Nexus Inventory.